



EMERGENT PROMPTS

5 Security Checks Before You Launch Your App

Copy-paste prompts that make Emergent find and fix the security mistakes that get vibe-coded apps hacked — before they go live.

MAYANK SHAH
@mayankshah_ai

Read this before you paste anything

Each of the five prompts below is based on a real open-source security tool used by professional engineers. Paste them into Emergent one at a time — in order — and let it audit and fix your app. Every prompt ends by asking Emergent to report exactly what it found and changed, so you can verify the work instead of trusting it blindly.

Run them in order. Prompt 1 catches leaked secrets, 2 audits personal data, 3 is the pre-deploy checklist, 4 goes deep on payments and auth, and 5 attacks your app like a hacker would.

Edit the brackets. Prompt 4 has a bracketed line you must customize to describe your app before pasting. Everything else works as-is.

Read the summaries. Don't just paste and move on. Each prompt makes Emergent list what it fixed — skim that list so you understand your own app's weak points.

One honest note: these prompts catch the mistakes that cause most real-world breaches in vibe-coded apps — but no AI audit is a substitute for professional security testing. If your app handles real money or sensitive user data at scale, run these and get a human security review before launch.

THE FIVE CHECKS

- | | | |
|----|---------------------------------------|-------------------------------|
| 01 | Secret Leak Prevention | based on Gitleaks |
| 02 | Personal Data Flow Audit | based on Bearer |
| 03 | Pre-Deploy Production Audit | based on ECC Production Audit |
| 04 | Deep Security Audit for Complex Logic | based on Trail of Bits Skills |
| 05 | Attacker's Perspective Review | based on ECC Security Review |

01 PROMPT 1 Secret Leak Prevention

Based on: Gitleaks — github.com/gitleaks/gitleaks

Finds and fixes hardcoded passwords, API keys, and tokens in your code before they get exposed publicly.

PASTE THIS INTO EMERGENT ▸

Before deploying this app, do a full secret safety pass across the entire codebase. Here's exactly what to check and fix:

1. Move all secrets to environment variables. Find every API key, password, token, database URL, and credential in the code. No secret should exist as a string literal anywhere in the source code — not in config files, not in utility functions, not in comments, nowhere.
2. Check these specifically: Supabase keys — the anon key is designed to be used client-side, but ONLY if Row Level Security is enabled on every table; without RLS it exposes your entire database. The service role key must NEVER appear in client-side code under any circumstances. Stripe keys (publishable AND secret) — only the publishable key goes client-side; the secret key stays server-side only. Database connection strings (MongoDB URI, PostgreSQL URL) — environment variable only, never hardcoded. OAuth client secrets and JWT signing secrets — server-side only. Any third-party API key (OpenAI, SendGrid, Twilio, Firebase, AWS) — all must be in env vars.
3. Check frontend exposure. React/Next.js exposes any env var prefixed with NEXT_PUBLIC_ or REACT_APP_ to the browser — make sure no sensitive key uses these prefixes. Only public-safe values (like a Supabase anon key WITH RLS enforced) should be exposed to the client.
4. Gitignore and .env.example. Make sure .env is in .gitignore. Create a .env.example file that lists all required variables with placeholder values but no real secrets.
5. Check logs and responses. Check console.log, error handlers, and API responses — make sure none of them accidentally print or return secrets, tokens, or connection strings.
6. Git history warning. If any secret was previously hardcoded, that old value is still in git history. Add a warning in README: rotate any previously hardcoded secrets immediately.

Show me a summary of every secret you found, where it was, and what you moved it to.

02 PROMPT 2 Personal Data Flow Audit

Based on: Bearer — github.com/bearer/bearer

Tracks how your users' personal data (emails, phones, passwords) moves through the code and makes sure it doesn't end up in logs, third-party services, or places it shouldn't be.

PASTE THIS INTO EMERGENT ▸

Do a full audit of how user personal data moves through this app. I need to know exactly where sensitive data enters, where it travels, and where it ends up.

1. Map all data collection points. Find every place the app collects user data — emails, phone numbers, passwords, names, addresses, dates of birth, payment info, IP addresses, device info. For each one, trace where that data goes after collection.
2. Clean all logs. Check every console.log, logger, print statement, and error handler. If ANY of them output user emails, passwords, phone numbers, tokens, or any personal data — remove that data immediately. Replace with "[REDACTED]" or remove the log entirely.
3. Audit third-party integrations. Check every third-party API integration and SDK (analytics, error tracking, payment processors, email services, AI APIs). For each one, list exactly what user data is being sent. Strip any extra fields the service doesn't need.
4. Password handling. Passwords must be hashed (bcrypt, argon2, or scrypt — never MD5 or SHA256 alone) before storage. Plaintext passwords should never be stored, logged, returned in API responses, or sent anywhere other than the hashing function.
5. Cookie and storage audit. If sensitive data is stored in cookies, make sure cookies have httpOnly, secure, and sameSite flags. User PII should not be in localStorage — it's accessible to any JavaScript on the page, including XSS attacks.
6. API response filtering. No endpoint should return more user data than the client needs. Implement field-level filtering on every response. No password hashes, no internal IDs, no other users' data.
7. Data deletion. Check if there's a way for users to delete their data. If not, add a basic account deletion flow that removes or anonymizes all personal data.

Show me a complete map: what data is collected, where it's stored, where it's sent externally, and what you fixed.

03 PROMPT 3 Pre-Deploy Production Audit

Based on: ECC Production Audit — github.com/affaan-m/ECC/tree/main/skills/production-audit

Checks everything that needs to be right before your app goes live — env vars, debug code removal, error handling, security headers, rate limiting.

PASTE THIS INTO EMERGENT ▸

This app is about to be deployed. Before it goes live, run through every single one of these checks and fix anything that fails:

1. Environment variables. Verify that every env var the app needs is referenced properly and has a fallback or clear error message if missing. The app should refuse to start if a critical variable (database URL, API keys, auth secret) is not set.
2. Debug code removal. Find and remove: console.log used for debugging, commented-out code blocks, TODO/FIXME comments referencing incomplete security features, hardcoded test credentials, any test-only endpoints (/test, /debug, /admin-backdoor, /seed-data). Debug mode must default to OFF.
3. Error handling. No error response sent to the client should include stack traces, database query details, file paths, or internal server info. Errors should return a generic message and a correlation ID. Detailed errors go to server-side logs only.
4. Security headers. Add to every response: X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Strict-Transport-Security (max-age 1 year), Content-Security-Policy restricting scripts to your domain. If using Express, use helmet middleware.
5. Rate limiting. Authentication endpoints (login, signup, password reset, OTP) must have rate limiting. Minimum: 5 attempts per minute per IP on login, 3 per hour on password reset.
6. CORS configuration. Check that CORS is not set to allow all origins (*) unless the API is genuinely public. Restrict to your specific frontend domain.
7. Database security. Database connection must use TLS/SSL in production, no default credentials, no open database port exposed to the internet without authentication.

List every check, whether it passed or failed, and what you fixed.

04 PROMPT 4 Deep Security Audit for Complex Logic

Based on: Trail of Bits Skills — github.com/trailofbits/skills

Professional-grade security audit for apps with payments, custom auth, smart contracts, or complex server logic.

▲ Before pasting: edit the bracketed line in the first sentence to describe YOUR app — e.g. "This app has Stripe payments and custom email/password auth." Delete the brackets.

PASTE THIS INTO EMERGENT ▸

This app has [payments / custom auth / complex server logic]. Do a deep security audit on the critical paths:

AUTHENTICATION & AUTHORIZATION: Check every protected route and API endpoint for proper auth middleware. Verify there's no IDOR — no endpoint should accept a user ID from the client and return that user's data without checking ownership. Check the password reset flow — tokens must be random, single-use, time-limited (15 min max), and tied to a specific user. Check JWT handling — strong signing secret, expiry, token blacklist on logout.

PAYMENT LOGIC: Never trust client-side price calculations — the server must independently calculate totals, taxes, and discounts. Check if an attacker could modify price/quantity/discount in the request body. Verify webhook signatures from payment providers (Stripe, Razorpay). Check that payment status is verified server-side before granting access to paid features.

INPUT HANDLING: Check every form input, URL parameter, and API field for SQL injection — replace any raw SQL with parameterized queries. Check for XSS — does any user input get rendered in HTML without sanitization? Check file uploads — validate file type server-side, limit file size, don't serve uploads from the same domain with executable permissions.

For every issue found, show me: what the vulnerability is, where it is in the code, how an attacker would exploit it, and the exact fix.

05 PROMPT 5 Attacker's Perspective Review

Based on: ECC Security Review —

github.com/affaan-m/ECC/tree/main/.agents/skills/security-review

Looks at your app the way a hacker would — finding every way to gain unauthorized access, steal data, or break the logic.

PASTE THIS INTO EMERGENT ▸

Think like an attacker trying to break this app. Check these specific attack paths:

1. Data access via ID manipulation. Can I access another user's data by changing an ID in the URL or request body? Try every endpoint that takes a user ID, order ID, or document ID — check if the app verifies ownership before returning data.
2. Login bypass. Check if any API endpoint works without an auth token. Check if the app validates expired or malformed tokens properly. Check for default admin accounts with known credentials.
3. Privilege escalation. If the app has roles (user, admin, moderator), check if a regular user can access admin endpoints by guessing URLs or modifying their role in the JWT/session. Role checks must happen server-side, not just by hiding UI elements.
4. Feature abuse. Check rate limits on: signup (mass account creation?), messaging (spam?), file uploads (storage fill?), API calls (DDoS?), promo codes or referral systems (infinite use?).
5. Content injection. Try putting JavaScript in every text field — usernames, bios, comments, search bars, file names. Check for SQL injection through search fields, filters, and login forms.
6. Internal exposure. Check if any of these are exposed: database admin panel, env vars through error messages, .env file via direct URL, .git directory, Swagger/OpenAPI docs that should be internal-only, health check endpoints leaking system info.
7. Business logic manipulation. If payments exist — can I pay negative amounts? Stack discounts infinitely? Restart free trials? Refer myself? These are logic flaws, not code bugs.

For every vulnerability found: explain what an attacker would do, how much damage they could cause, and fix it immediately. Data theft and unauthorized access first, abuse and logic flaws second.

You're done — almost. Re-run Prompt 5 after any major feature update: new code means new attack surface. All GitHub tools referenced in this guide are open-source, free to use, and were verified live at the time of publishing. And remember — for apps handling real money or sensitive data at scale, pair these prompts with a human security review.

Instagram — @mayankshah_ai · YouTube — @mayankshah_ai